

An explainable efficient flow-based Industrial IoT intrusion detection system[☆]

Mohammed M. Alani^{*}

Cybersecurity Research Lab, Toronto Metropolitan University, Toronto, Canada

School of IT Administration and Security, Seneca College of Applied Arts and Technology, Toronto, Canada

ARTICLE INFO

Keywords:

IIoT
IoT
Security
Intrusion detection
Industrial IoT
Explainable ai

ABSTRACT

With the steady growth in the adoption of IoT in an industrial context, security challenges intensify. Security threats to IoT devices in a nuclear plant or an oil refinery are much higher risk compared to home appliances. In this paper, we present an explainable, efficient, and highly accurate machine-learning based intrusion detection system for industrial IoT, referred to as E2I3DS. This work successfully reduces the needed features within the WUSTL-IIOT-2021 dataset from 48 features to 11 only, while maintaining very high accuracy. Experiments utilizing the 11-feature dataset showed that proposed system has a higher accuracy of 99.97%, with a false positive rate of 0.01% and a false negative rate of 0.04%. The proposed system also proved to be very efficient with an excellent detection time of 0.1517 μ s. In addition, the model was explained using Shapley additive explanations.

1. Introduction

Internet of Things (IoT) is the proliferation of smart devices such as tablets, phones, TVs, sensors, etc. IoT brought many benefits to home users, such as reducing energy and time consumption, among other benefits. In an industrial context, Industrial IoT (IIoT) is designed to handle heavy-duty tasks such as manufacturing, monitoring, etc. In principle, IIoT devices are usually more precise and durable (heat/cold resistant) devices compared to home IoT devices.

Both the IoT and the IIoT have the same core principles such as data management, networking, sensors, actuators, etc. The main differences between IoT and IIoT are scalability, volume of generated data, and how this data is handled. In general, IIoT devices generate a massive amount of data. Thus, IIoT makes a good candidate for big data and machine learning applications. Another major difference between IoT and IIoT is that most home IoT devices are focused on actuating, rather than sensing. Hence, the data generated by most home IoT devices go unnoticed. On the other hand, data generated by IIoT devices are vital to the manufacturing process, and thus need to be more precise and continuous, in most cases. For instance, considering a monitoring system in a nuclear power plant or a manufacturing facility, it should be highly precise and continuous to avoid disastrous events [1].

IIoT is seeing rapid growth in adoption worldwide. Fig. 1 shows the IIoT market size projections from 2017 to 2025. With an almost 90B USD market in 2022, the growth looks both steady and massive.

This rapid growth in the adoption of IIoT worldwide makes it a desirable target for malicious actors. IIoT introduces new attack surfaces and intensifies security challenges. IIoT is an important part of Industry 4.0 and beyond. Therefore, protecting IIoT devices

[☆] This paper was submitted for special section VSI-aips, but should be included in regular issues of CAEE. Reviews were processed by Guest Editor Prof. Ali Kashif Bashir and recommended for publication.

^{*} Correspondence to: School of IT Administration and Security, Seneca College of Applied Arts and Technology, Toronto, Canada.

E-mail address: m@alani.me.

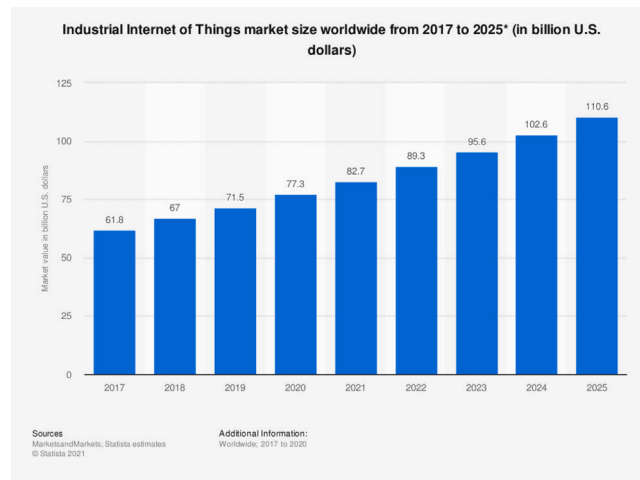


Fig. 1. Industrial Internet of Things market size worldwide from 2017 to 2025 [2].

is an inescapable need. Integrity and security concerns for IIoT when compared to those in IoT are more connected to safety-critical and mission-critical factors.

A challenge that IIoT faces is the low vendor commitment to security. Many vendors do not issue appropriate patches to known vulnerabilities on their devices. This, combined with the hesitation to apply these patches by device users fearing to break the device or to break something else in their systems, makes IIoT devices a high-risk asset and an interesting target for malicious actors.

According to [3], the following list shows the factors that influence IIoT security:

1. **Business Goals and the Use of Technology:** The adoption of IIoT must align with the business goals of the organization. In most cases, security is seen as an obstacle that prevents the organization from achieving its goals smoothly. This makes management less eager to adopt proper security practices.
2. **Human Factors:** Human factors play an important role in the design and implementation of IIoT systems. With the major focus of attackers on social engineering, the human factor also plays an important role in security. Malicious actors often start their attacks with different social engineering techniques, such as phishing emails or phishing URLs.
3. **Human Safety:** As many IIoT systems handle sensitive processes such as drinking water filtration, or nuclear power plants, its breach is considered of extremely high risk unlike IoT systems in other contexts such as smart home IoT devices.
4. **Compliance and Regulations:** The new regulations and rules proposed require the examination of the IIoT design and implementation concepts to ensure compliance. These regulations are expected to play an important role in the future of IIoT.
5. **Data Storage Location:** IIoT devices, by nature, produce large amounts of data. Protecting these data in transit and at storage is a complicated task. Many questions arise about data storage; should the data be stored on premises or outside? should it be stored in the cloud? Would cloud storage be in compliance with the regulations? among many other questions.
6. **Industrial Sabotage:** With the introduction of IIoT as a new attack surface, this technology becomes a playground in which malicious competitors can perform industrial sabotage and espionage.
7. **Real-Time Process Control:** With the rapidly increasing interest in remote process control, the compromise of any component in the path of industrial process control messages can be a serious security threat.

Although machine learning was utilized in intrusion detection for many years [4,5], research in intrusion detection within the industrial IoT context did not witness a similar growth. Many articles proposed IIoT intrusion detection while utilizing non-IoT datasets. These articles ignore the uniqueness of the traffic attributes, and attack types within the IIoT context, as characterized by the factors listed above. In addition, most provided ML-based solutions are presented as black-boxes with no proper explanations on how these systems make their detection decisions. Another factor that contributes to the research gap is the lack of a unified feature set that can capture different types of attacks [6–8].

1.1. Research contribution

The proposed system addresses several gaps in the research such as the explainability of IIoT intrusion detection, the coverage of a wider range of attacks by utilizing a newer dataset, and the focus of the dataset on IIoT, specifically. In addition, the proposed work has improved efficiency due to utilizing a much smaller number of features.

The contributions of this research can be summarized in the following points:

Table 1
Summary of previous works.

Work	Contribution	Accuracy	Explainable ML
Kim and Kang [9]	Signature-based Modbus attack detection	–	×
Wallis et al. [10]	Rule-based context attributes in deep packet inspection	–	×
Zolanvari et al. [11]	IIoT intrusion detection using WUSTL-IIOT2018 dataset	99.9%	×
Yao et al. [12]	A conceptual hybrid ML-assisted intrusion detection for edge-based IIoT	–	×
Abdel-Basit et al. [13]	Using Local-gated recurrent networks to detect attacks in BoT-IoT	99.75%	×
Hassan et al. [14]	Using deep learning for adaptive-trust boundary protection for IIoT	96.63%	×
Mahrani et al. [15]	ML-based attack detection in fog computing using KDD'99 dataset	93.3%	×
Teixeira et al. [16]	Real-time intrusion detection based on neural networks	–	×
Song et al. [17]	SDN-based IIoT attack detection using machine learning	99%	×
Awutunde et al. [18]	DNN-based intrusion detection with genetic rule-based feature selection	98.9%	×
Vulfin et al. [19]	Traffic analysis using DT classifier for intrusion detection on WUSTL-IIOT-2018	9.99%	×
Kasongo [20]	Intrusion detection using RF classifier with genetci algorithm-based feature selection using UNSW-NB15	87.61%	×
Alani et al. [21]	Explainable DNN-based intrusion detection using WUSTL-IIOT-2021	99.94%	✓
Dina et al. [22]	FNN-based intrusion detection system using WUSTL-IIOT-2021	98.94%	×

1. This paper presents an efficient and highly-effective machine learning-based intrusion detection system (IDS) trained on IIoT-specific data.
2. The proposed system uses recursive feature elimination (RFE) to create an 11-feature version of the original 48-feature IIoT traffic dataset named WUSTL-IIOT-2021.
3. The selected features and the trained classifier are explained using Shapley additive explanation (SHAP).

1.2. Paper layout

The following section will provide a review of previous works that addressed the same problem. Section 4.3 explains the details of the selected dataset; WUSTL-IIOT-2021. An overview of the proposed system along with the details of the detection processes are explained in Section 3. Section 4 shows the details of the experiment, starting from pre-processing, to initial results, and ending with cross-validation results. Section 5 details the model's explanation with a brief introduction to SHAP values and how to interpret them. The results are discussed in Section 6. The last section of the paper draws conclusions and suggests future research directions.

2. Previous works

Kim and Kang proposed, in 2018, an attack detection scheme for IIoT that is focused on Modbus protocol [9]. The proposed mechanism detects abnormal traffic by tracing Modbus/TCP transactions. The proposed mechanism detects Modbus induced Denial of Service (DoS) attacks along with nonmalicious incidents such as routing loops, misconfigured devices, and human errors. The proposed method uses three detection tables to detect abnormal traffic on Modbus/TCP transactions. Each table has information about tracing and managing the Modbus/TCP request and response messages. These three tables, the Command Table, Transaction Table, and Exception Table, contain specific signatures of different types of attacks that are matched with the incoming traffic to perform the detection process. The paper does not address a wide range of attacks and is limited to the signatures included in the three tables mentioned earlier.

In 2018, Wallis et al. proposed a rule-based approach that uses subcontexts from the IIoT context for deep packet inspection to protect the privacy of data and to increase the resilience of the system against attacks [10]. The proposed work compiles each group of attributes into a subcontext, an compiles each group of subcontexts into a single IIoT context. The study develops formal definitions for the Industrial Internet of Things context, subcontexts, and context rules. The study also proposes the use of the defined subcontexts in the construction of a new protocol called context enhancement protocol, to add context information to packets and utilize these context information in detection of anomalies. As any other rule-based intrusion detection system, it fails to detect unknown attacks and zero-day attacks.

Zolanvari et al. published a study in 2018 that addressed the impact of imbalanced dataset on machine-learning based security solutions for IIoT [11]. The study discusses how IIoT networks have fundamental differences compared to regular corporate networks. The study mentions that the vulnerabilities and security requirements of IIoT systems require different considerations. The study examines why machine learning must be integrated into the security mechanisms of the IIoT and where it currently falls short. The study also discusses the challenges and real-world considerations associated with this matter. The study presents a testbed that is close to real-world IIoT scenarios that can be used in future studies. In 2019, the authors presented a continuation of their previous study through another comprehensive study where they introduced a more comprehensive version of the testbed where the older version of the dataset that we are utilizing in this study, referred to a WUSTL-IIOT-2018 [23]. The testing of the proposed IDS system yielded an accuracy of 99.99%, based on 25 features of the network flow. The dataset included three attack types; SQL injection, command injection, and backdoor attacks.

Yao et al. presented, in 2019, a hybrid IDS architecture with a machine learning-assisted detection method [12]. The focus of the research was on intrusion detection for edge-based IIoT. The study divides edge-based IIoT to devices with strong computing power and sufficient resources such as edge routers, which serve as master nodes, while the industrial equipment of the edge part is

regarded as edge nodes. On edge nodes, light-weight LightGBM algorithm is applied to perform detection process and to extract more advanced features to represent the traffic without increasing the resource consumption. On master nodes, deep learning is applied to perform the second intrusion detection task. This research was more of a conceptual paper and did not have an implementation part.

In 2020, Abdel-Basset et al. proposed a deep learning model based on forensics (called Deep-IFS) to identify intrusions in IIoT traffic [13]. The model learns local representations using local gated recurrent unit (LocalGRU) and introduces an MHA layer to capture and learn global representation (i.e., long-range dependencies). A residual connection between layers was proposed to prevent information loss. The proposed model was deployed and trained in a fog computing environment. The computations and IIoT traffic data are distributed across worker fog nodes for training, and the master fog node shares training parameters and aggregating worker node output. The aggregated classification output is subsequently passed to the cloud platform for mitigating attacks. The proposed model was tested using the Bot-IoT dataset and achieved accuracy of 0.9975, and F_1 score of 0.9987. The proposed model is said to be designed specifically for IIoT setup. However, the dataset used is a general IoT dataset for Botnet traffic.

Hassan et al. proposed, in 2020, an adaptive trust boundary protection for IIoT networks using a deep-learning, feature-extraction-based semi-supervised model [14]. The proposed model is said to be compatible with the multilevel protocols of IIoT. The proposed approach can learn the nature of unknown attack models using unsupervised learning and unlabeled data from the wild. This makes the proposed approach resilient to emerging cyberattacks and their dynamic nature. The testing showed that the proposed system achieved an accuracy of 0.9663 when using an RF classifier with neural network of the architecture 1024-1024-512, which is quite resource intensive.

In 2020, Mahrani et al. proposed an attack detection mechanism in the fog computing framework with several machine learning algorithms to efficiently detect malicious activities [15]. The proposed method was evaluated using the KDD Cup'99 dataset, which is a general intrusion detection dataset that does not include IoT devices, not IIoT devices, and compared using Decision Tree, K-Means, and Random Forest algorithms. K-Means classifier achieved the best accuracy of 0.9333 only. The study did not utilize a dedicated IIoT intrusions dataset, and hence the results are not reliable for IIoT applications.

Teixeira et al. presented, in 2021, a neural network-based online intrusion detection system [16]. The proposed IDS used 19 network-flow features with the focus on real-time detection. The testing showed that the proposed system achieved an accuracy of 99.98%. However, the study did not measure efficiency or timing metrics although it proposes a real-time solution.

In 2021 as well, Song et al. presented a Software-Defined Network (SDN)-based industrial internet security gateway to protect IIoT [17]. The proposed system combines the Extratrees Classifier to enhance the detection rate of anomaly detection. This paper simulates an industrial network attack using the acquired training data for testing. The test data are industrial network traffic datasets, and the experimental results showed an accuracy of 0.99.

Awutunde et al. presented, in 2021, an industrial IoT intrusion detection system based on deep neural networks [18]. The proposed model used genetic rule-based feature selection in combination with deep neural networks. The proposed system was trained and tested using NSL-KDD and UNSW-NB15 datasets, to achieve an accuracy of 98.9%. However, both of the utilized datasets are general intrusion detection datasets and do not include IIoT traffic.

Vulfin et al. presented, in 2021, a network traffic analysis system using machine learning models [19]. The study carried out the analysis of the dataset WUSTL-IIOT-2018. The study also discusses the possibility of embedding models as modules of specialized network equipment, which makes it possible to increase the efficiency of the analysis of network traffic, including specialized industrial protocols. Testing showed 0.9999 accuracy for the proposed DT classifier.

Kasongo proposed, in 2021, an IDS for IIoT that employs Genetic Algorithm (GA) for feature selection, and Random Forest (RF) model as a fitness function [20]. The models used for intrusion detection processes included RF, Linear Regression (LR), Naïve Bayes (NB), Decision Tree (DT), Extra-Trees (ET), and Extreme Gradient Boosting (XGB). The GA-RF generated 10 feature vectors for the binary classification scheme and 7 feature vectors for the multiclass classification procedure. The dataset used for training and testing was UNSW-NB15, which is a general IDS dataset that is not collected from IoT, nor IIoT devices. The proposed system presented an accuracy of 87.61%.

In 2022, Alani et al. proposed a deep learning-based IIoT intrusion detection system utilizing a multi-layer perceptron neural network [21]. The proposed system employed the full 41-feature WUSTL-IIOT-2021 dataset. While, the proposed model achieve a noticeable high accuracy of 99.94%, employing neural networks in the IIoT environment could be a taxing practice due to the high calculations intensity.

Dina et al. proposed, in 2023, an intrusion detection system that trains neural-networks using focal loss function [22]. The proposed system was tested using three different datasets, including WUSTL-IIOT-2021 that we used in our research. While the proposed system delivered a high accuracy of 98.94%, its F_1 score was noticeably low at 0.6848. This comes from the fact that the authors did not address the significant imbalance in the different classes of the dataset. The model ended up being well trained for one class, and performed badly in the other class.

A summary of the reviewed previous works can be found in Table 1.

3. Proposed system overview

The design principles of proposed system are high accuracy, high efficiency, ease of implementation, and explainability. High accuracy is achieved by proper preprocessing that prepares the data for training, and utilizing different types of machine-learning classifiers, and choosing the one with the highest accuracy to deploy. Another contributing factor to high accuracy is the reduction

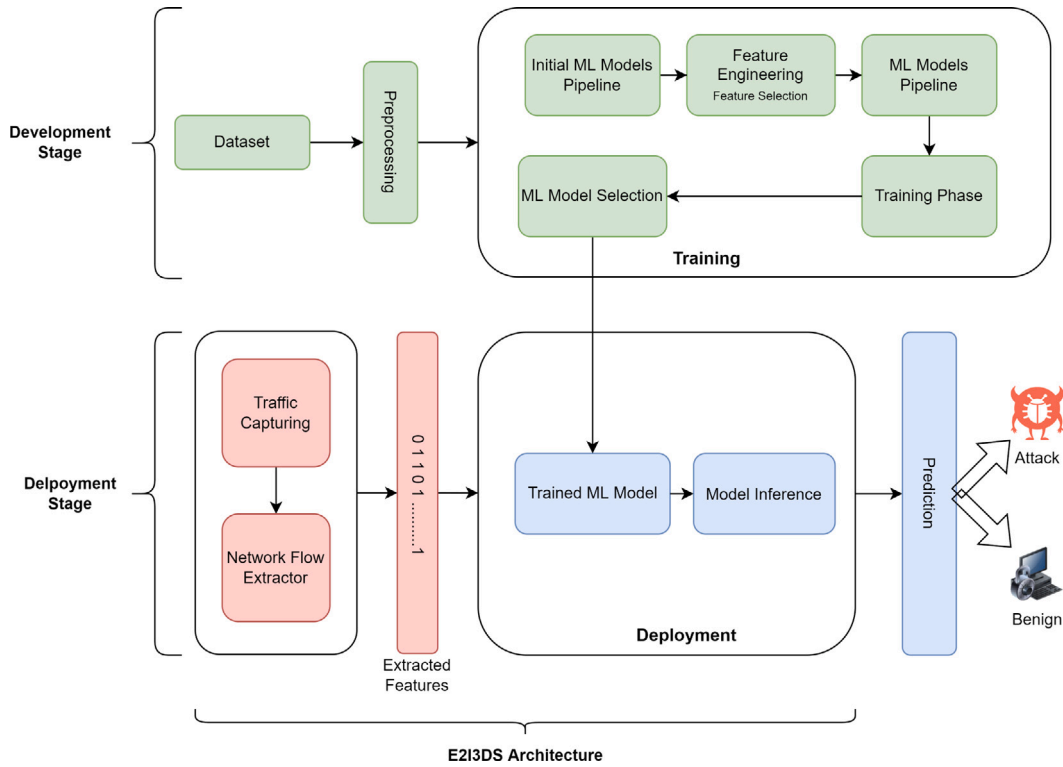


Fig. 2. An overview of the proposed system (E2I3DS).

of the number of features from 41 to 11. This eliminates noisy features that impact the decision of the classifier negatively. This large reduction impacts not only the training time, but also impacts the prediction time noticeably. In addition, fewer number of features translates to less time required to extract and the features in real-life deployments, and lower memory and processing requirements. Implementability is achieved by following RFE using feature importance to select the smallest number of features that can help the classifier achieve the desired accuracy. By using this method, rather than a classical dimensionality reduction methods such as principal component analysis, the real-life deployment of the system will require minimal preprocessing before feeding the data into the classifier. In addition, all the selected features can be easily extracted from network-flow information. The fourth design goal, i.e. explainability, is achieved through using SHAP explanation to get a better understanding of the decisions made by the classifier.

Fig. 2 shows an overview of the proposed system.

As shown in Fig. 2, the system is divided into two stages; development stage, and deployment stage. At the development stage, the dataset is preprocessed and fed into the training unit. At the first stage of training, four different classifiers are created to help select the best performing classifier so that it would be used in the feature selection process. For our experiments, we picked the following classifiers:

- Random Forest (RF)
- Logistic Regression (LR)
- Decision Tree (DT)
- Gaussian Naive Bayes (GNB)

These classifiers were picked because they have shown persistent strong performance in intrusion detection as shown in [24] and [25].

Performance indicators obtained from this initial training and testing dictates which classifier to be used in the following feature selection stage. As our proposed feature-selection method depends on feature importance, identifying the best performing classifier type will help us identify the feature importance calculation method. The dataset undergoes feature selection phase where a smaller number of features is selected to produce a reduced version of the dataset to facilitate more efficient processing. The reduced version of the dataset is then used to train and test a new set of classifiers to measure the impact of the feature selection process and ensure that the selection did not negatively impact the performance. This new set of classifiers utilizes RF, LR, DT, and GNB algorithms. Then, the best performing classifier undergoes further validation to ensure that it can generalize well beyond its training dataset. Once that is done, the chosen classifier is stored to be used at the deployment stage.

At the deployment stage, network traffic is captured using a tool named tshark, and the network-flow features are extracted using a tool named Zeek, and forwarded to the trained classifier. The classifier performs inference to produce a prediction whether

Table 2
Hardware specifications of the implementation environment.

Processor	AMD Ryzen 5 3600
Clock Speed	4.2 GHz
RAM	128 GB
GPU	NVidia GeForce RTX3060Ti

Table 3
Software specification of the implementation environment.

Operating System	Windows 10 Professional
Python	3.8.5
Sci-Kit Learn	1.0.1

this network-flow is classified as “attack” or “benign”. This prediction can be forwarded to the device’s operating system to block the source IP address of the attack, and log the incident.

4. Implementation and results

The implementation section contains description of the implementation environment, description of the experiment design, definition of the performance metrics used in this study, preprocessing details, initial training and results, features selection, and cross-validation process and results.

4.1. Implementation environment

Tables 2, and 3 show the hardware and software specifications of the implementation environment that was used in preprocessing, training, testing, and storage of the trained model.

4.2. Performance metrics

The basic four performance metrics of a binary classifier, according to [26], are True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). These four metrics, when combined together, generate the confusion matrix.

In our research, we will be using the following six performance metrics:

1. Accuracy

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (1)$$

2. Precision

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

3. Recall

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (3)$$

4. F_1 score

$$F_1 \text{ score} = 2 * \frac{\frac{\text{TP}}{\text{TP} + \text{FN}} * \frac{\text{TP}}{\text{TP} + \text{FP}}}{\frac{\text{TP}}{\text{TP} + \text{FN}} + \frac{\text{TP}}{\text{TP} + \text{FP}}} \quad (4)$$

5. Training Time

The time required to train the classifier using the training subset.

6. Testing Time

The time required for the trained classifier to process one input instance and produce a prediction.

Table 4
Specifics of WUSTL-IIOT-2021 dataset.

Number of samples	1,194,464
Number of features	41
Number of attack samples	87,016
Number of normal samples	1,107,448

4.3. The dataset

The dataset used in training and testing the proposed system was WUSTL-IIOT-2021 [27]. The dataset characterizes network-flow data collected from IIoT devices. The raw data was collected from the IIoT testbed presented in [28]. This testbed was designed to emulate real-world industrial networks. The testbed includes IIoT systems that supervises the water level and turbidity quantity of the water storage tank. This system is a part of the water treatment and distribution process in industrial reservoirs. This testbed includes three sensors and four actuators.

The raw data collected was 2.7 GB of data over the span 53 h. Raw packets were collected from the testbed, and then used to extract network-flow features from them using a tool named Argus. Each sample included 41 features extracted to represent one network flow. Table 4 shows the statistical details of the dataset. The network-flow features were chosen by the authors by analyzing the features whose values change during the attack phases compared to the normal operation phases, according to [27]. The full list of features can be found in Appendix.

From Table 4 we can see that the “benign” traffic represents about 92% of the dataset, while the “attack” samples constitute about 8% of the dataset only.

Attack samples included the following types of attack:

1. Command Injection Traffic (0.31%)
2. DoS Traffic (89.98%)
3. Reconnaissance Traffic (9.46%)
4. Backdoor Traffic (0.25%)

4.4. Dataset preprocessing

Upon deep examination of the dataset, we noted the following observations:

- There is large imbalance between the “attack” and “benign” classes, as the “attack” class represents less than 8% of the dataset.
- There is a large imbalance between the specific attack types, where DoS attack represent almost 90% of the attack class, and backdoor traffic represents less than 0.25% of the attack class.
- The existence of noisy features that are not relevant to the attacks such as StartTime, and LastTime.
- The existence of host-specific features such as SrcAddr, DstAddr, sIpId, and dIpId. If not removed, these features can negatively impact the generalization of the model significantly.

Based on the above mentioned observations we constructed our preprocessing procedure, as shown in Algorithm 1.

Algorithm 1: Dataset Proprocessing

Input: Original Dataset (1,194,464 instances, 48 features)

Output: Balanced Dataset with no missing data (261,048 instance, 41 features)

Array $\leftarrow$ *RawDataset*

In (*Array*) remove *traffic* label

In (*Array*) remove *StartTime*, *LastTime*, *SrcAddr*, *DstAddr*, *sIpId*, *dIpId* features

Array $\leftarrow$ *RandomUnderSampling_MajorityClass(Array)* *Dataset* $\leftarrow$ *Array*

As shown in Algorithm 1, the process starts by removing the specific attack type label. As the goal of the research is to create an intrusion detection system, and not classifying the specific type of attack, the attack label was removed. The damage caused by the large imbalance of different attack types outweighs the benefits of attack classification, as attack classification is beyond the scope of this work.

In the second step, the source- and destination-specific features that were mentioned in the original dataset were removed to ensure better generalization of the classifier.

The third step of preprocessing addresses the large imbalance between the “attack” and “benign” classes. To resolve the issue, we performed random undersampling to the majority class to reduce the number of instances from 1,107,448 to 174,032. This brought the total number of instances from 1,194,464 to 261,048. This makes the “attack” class 33.33% of the dataset size, and the “benign” class at 66.66%.

Table 5
Initial testing results with 41 features.

Classifier	Accuracy	Precision	Recall	F_1 score	Training time (s)	Testing time (μ s)
RF	0.9999	0.9999	0.9999	0.9999	21.8947	4.7767
LR	0.7992	0.8724	0.6987	0.7195	5.8438	0.1232
DT	0.9998	0.9998	0.9998	0.9998	2.0018	0.1534
GNB	0.9537	0.9583	0.9369	0.9465	0.1007	0.4944

4.5. Experiment design

The experiments of this research were divided into three stages:

1. Initial Model Training: In this stage, the classifiers' pipeline is trained and tested with the 41-feature dataset resulting from the preprocessing stage.
2. Feature Selection: At this stage, RFE is implemented to select a lower number of features.
3. A new pipeline of classifiers is created, trained, and tested using the reduced-features dataset to measure the impact of the feature selection process on the performance metrics.
4. 10-fold cross-validation: At this stage, final validation is performed on the best performing classifier to ensure its generalization beyond the training dataset.

The following subsections will explain the details of the experiments and their results.

4.6. Initial models training

As mentioned in Section 3, we chose four types of classifiers to train and test. At the initial training and testing phase, we used the dataset that was produced by the preprocessing process. This dataset included 41 features, 261,048 instances (174,032 "benign" instances, and 87,016 "attack" instances).

Table 5 shows the performance metrics produced by testing all the four classifiers, after training.

As shown in Table 5, RF classifier slightly outperforms DT in terms of accuracy, while DT performs about 10 times better in terms of testing time. LR performs poorly in comparison to the other three classifiers, while GNB performs slightly worse than RF and DT, in terms of accuracy. Based on slightly higher accuracy, we chose RF classifier to be the base for our feature selection process as explained in the following subsection.

4.7. Feature selection

The direction of this research is focused on generating a sub-dataset from WUSTL-IIOT-2021 that has a smaller number of features, while maintaining high accuracy when used to train machine learning models. This dictates that the selected features have to be explainable, and can be easily acquired during real-life data acquisition. This approach rules out certain statistical dimensionality reduction algorithms such as principal component analysis, singular value decomposition, and linear discriminant analysis in favor of feature selection algorithms that do not require intensive calculations in live deployments, such as RFE. The method that we used to select the features in this research was recursive feature elimination using feature importance. The reasons behind this choice was that dimensionality reduction methods mentioned earlier are resource-intensive, and require a large amount of preprocessing before having the features ready to be fed into the classifier in real-life deployments. Another reason of choosing RFE is that RFE does not only reduce the number of features fed into the classifier to improve efficiency, but also reduces the number of features that need to be acquired at the data acquisition stage of the deployment. RFE relies on a simple repetitive concept which is eliminate the feature with the lowest importance repeatedly while maintaining acceptable performance. The calculation of feature importance in algorithms such as DT, RF, or similar decision-tree based algorithms is based calculating the node impurity. The calculation finds the reduction in impurity of a node weighted by the number of samples that are reaching the node from the total number of nodes in the tree. In our work, we rely on the feature importance calculation function built into the SciKit-Learn package. The steps followed in RFE are shown in Algorithm 2.

As shown in Algorithm 2, we created a Random Forest classifier and trained it with 75% of the dataset samples, and tested it with the remaining 25% samples. Then, the feature importance is calculated for each feature, and the feature with the lowest feature importance is eliminated. Then, the process is repeated to eliminate the features with the lowest importance one by one through training, testing, and feature importance calculation. This process was repeated until the number of features reached 11. Our experiments showed that going below 11 features would cause a significant drop in the classifier's performance.

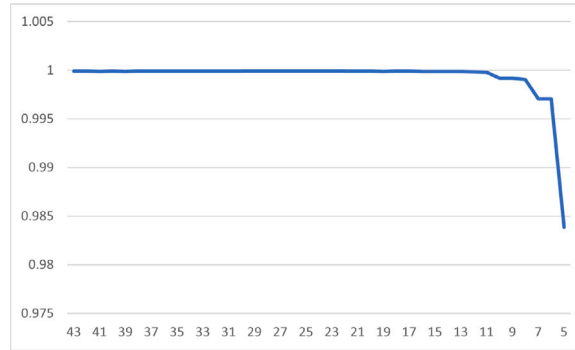
Zolnavari et al. in [23], relied on feature importance as well to select the features with the highest importance. In our research, we did not follow the same method of selecting the features with the highest importance. Instead, our proposed method relies on repetitive elimination of the feature with the lowest importance, and re-training the model again. We proposed this method because the features might be correlated and can be impacting each other's importance. This means that the importance of one features

Algorithm 2: Recursive Feature Elimination Using Feature Importance

Input: Dataset with 41 features
Output: Dataset with 11 features

$Array \leftarrow Dataset$
 $model \leftarrow RandomForestClassifier$
 $TargetFeatures \leftarrow 11$

while $Features(Dataset) > TargetFeatures$ **do**
 $RandomSplit(Array) \rightarrow Train_Array, Test_Array$
 train $model$ with $Train_Array$
 $importance \leftarrow FeatureImportance(model)$
 $i \leftarrow$ index of feature with lowest importance
 $Array.DeleteFeature(i)$
end
 $Dataset \leftarrow Array$

**Fig. 3.** The impact of feature reduction on F_1 score.**Table 6**
Testing results with 11 features.

Classifier	Accuracy	Precision	Recall	F_1 score	Training time (s)	Testing time (μ s)
RF	0.9998	0.9998	0.9998	0.9998	10.2043	4.4656
LR	0.6704	0.8351	0.5008	0.4029	0.2107	0.0001
DT	0.9997	0.9998	0.9997	0.9997	0.3503	0.1517
GNB	0.7817	0.8541	0.6742	0.6892	0.0468	0.1543

might be impacted by the existence (and therefore the elimination of) another feature. That is why RFE algorithm re-trains and re-calculates the importance after the elimination of each feature.

Fig. 3 shows the impact of recursive feature elimination on F_1 score of the trained and tested classifier.

As shown in Fig. 3, F_1 score dropped rapidly beyond 11 features. Hence, we stopped the reduction process at 11 features to prevent the degradation in model's performance.

4.8. Final training and testing

A new pipeline of classifiers was created, trained, and tested using the new dataset with the selected 11 features. Table 6 shows the performance metrics for testing the trained models with the 11-feature dataset.

As shown in Table 6, RF and DT still outperform LR and GNB in terms of accuracy. It is also noticeable that accuracy of RF, and DT declines only by 0.0001, which is considered insignificant. However, the impact of feature selection on LR and GNB's accuracy has been significant, which leaves them out of the competition with RF, and DT.

Fig. 4 shows the changes in performance metrics of RF and DT, and Table 7 shows the impact of feature reduction on timing parameters of RF and DT classifiers.

As shown in Fig. 4, the feature reduction did not have a significant impact on accuracy, precision, recall, and F_1 score, while timing parameters have significantly improved in both classifiers, as shown in Table 7. From these results, we can conclude that the feature selection was significantly successful in improving efficiency, while maintaining very high accuracy. Judging by accuracy and timing, the best performing classifier was DT, although RF has a slightly higher accuracy. The reason behind our choice is that testing time per instance in DT is about 3% of the testing time of RF.

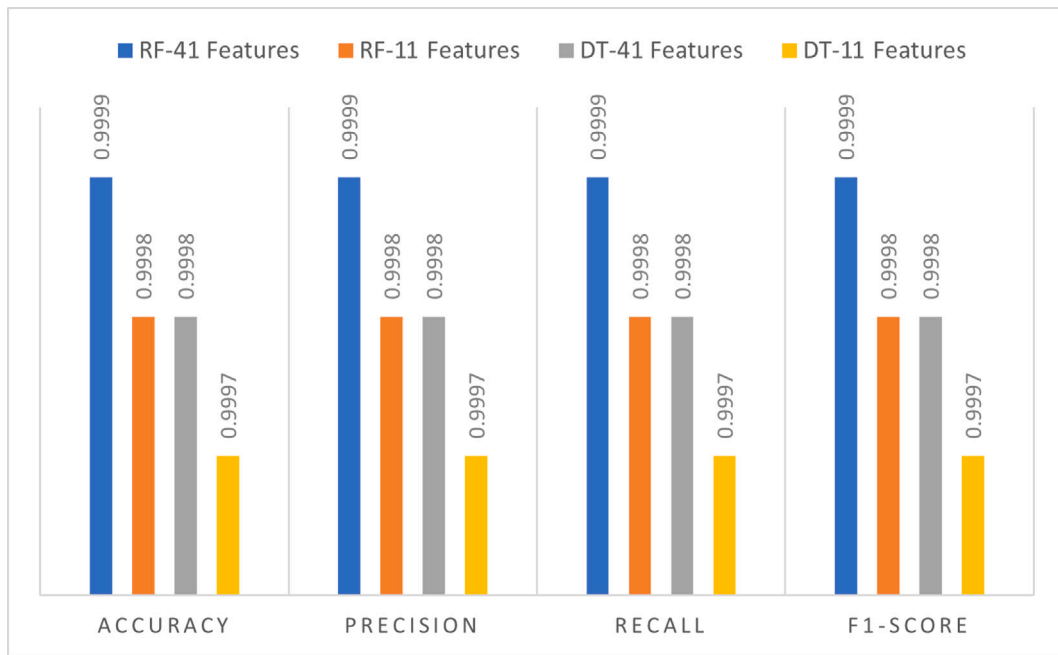


Fig. 4. Impact of feature reduction on performance metrics of RF and DT.

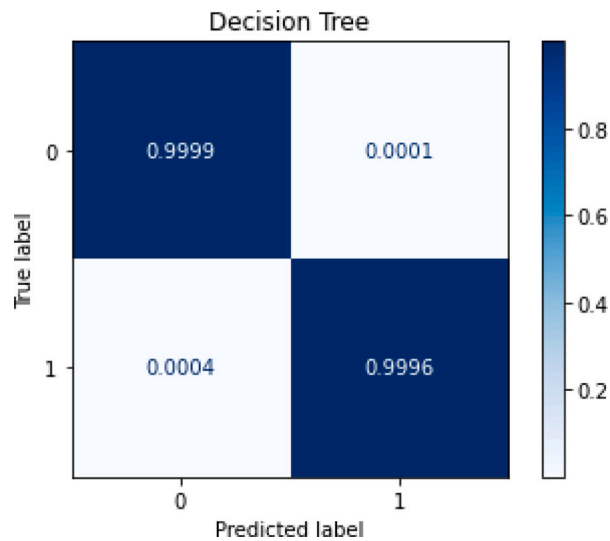


Fig. 5. Confusion matrix plot of DT classifier using 11 features.

Table 7

Impact of feature reduction on timing of DT and RF.

Model	Training time (s)	Test time (μ s)
RF-41 Features	21.8947	4.7767
RF-11 Features	10.2043	4.4656
DT-41 Features	2.0018	0.1534
DT-11 Features	0.3503	0.1517

Fig. 5 shows the confusion matrix plot of DT classifier when tested with the 11-feature dataset.

As shown in Fig. 5, DT classifier has recorded a FP rate of 0.01% only, while recording a FN rate of 0.04%. These two metrics are considered very good for an intrusion detection system as most systems suffer from high FP rates.

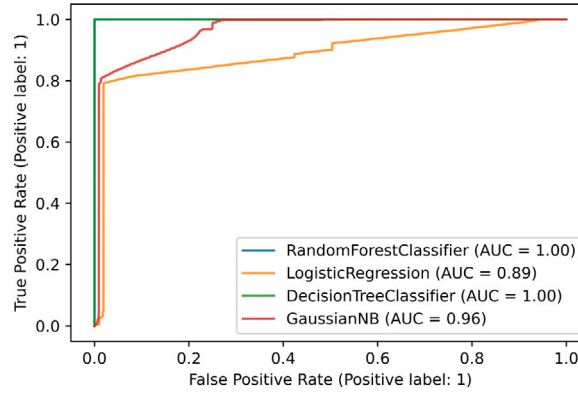


Fig. 6. ROC curve of the classifiers trained with the 11-feature dataset.

Fig. 6 shows the Receiver Operating Characteristic (ROC) curve for the classifiers trained using the 11-feature dataset. In this figure, DT and RF show an area under curve of 1.00.

4.9. 10-fold cross-validation

To ensure that the classifier generalizes well beyond its training dataset, we used 10-fold cross-validation on the 11-feature data, shown in Algorithm 3.

Algorithm 3: 10-fold Cross-Validation Algorithm

```

Input: Balanced dataset (data)
Output: Classifier performance parameters
create(classifier)
split_data(1to10) ← split_to_10_folds(data)
perf_m ← null
for i = 1to10 do
    train_data ← null
    j ← 1
    while j ≤ 10 do
        if j ≠ i then
            train_data ← append(train_data, split_data(j))
        end
    end
    train(classifier, train_data)
    test(classifier, data(i))
    perf_m ← performance_metrics(classifier)
end
print perf_m

```

In a 10-fold cross-validation process, the data is split into 10 subsets randomly. After the split, 9 subsets of the data will be used in training, and the 10th subset is used for testing. This process is repeated ten times until all subsets have been used in testing only once. The performance metrics obtained from the 10 runs would be compared together. If the measured metric suffer from high variance, then the model is suffering from over-fitting and is not generalizing properly within the dataset. If the variance is low, then the mean values of the performance parameters are reliable results.

Table 8 shows the results of the 10-fold cross-validation process along with the mean values of performance metrics from the 10 folds, and the standard deviation.

As shown in Table 8, accuracy has a very low standard deviation of 0.000072, while F_1 score standard deviation is 0.000109. These results mean that the variation is minimal, and that the classifier generalizes well beyond its training dataset. The mean accuracy was slightly higher than the testing in the previous subsection.

Table 8
Results of 10-fold cross-validation of DT classifier.

Fold	Accuracy	Precision	Recall	F_1 score
1	0.999808	0.999648	0.999765	0.999706
2	0.999923	0.999769	1.000000	0.999884
3	0.999962	1.000000	0.999885	0.999942
4	0.999923	0.999775	1.000000	0.999888
5	0.999847	0.999884	0.999652	0.999768
6	0.999732	0.999658	0.999544	0.999601
7	0.999962	1.000000	0.999887	0.999943
8	0.999962	1.000000	0.999885	0.999942
9	0.999923	0.999770	1.000000	0.999885
10	0.999885	0.999885	0.999769	0.999827
Mean	0.999893	0.999839	0.999839	0.999839
SDev	0.000072	0.000128	0.000147	0.000109

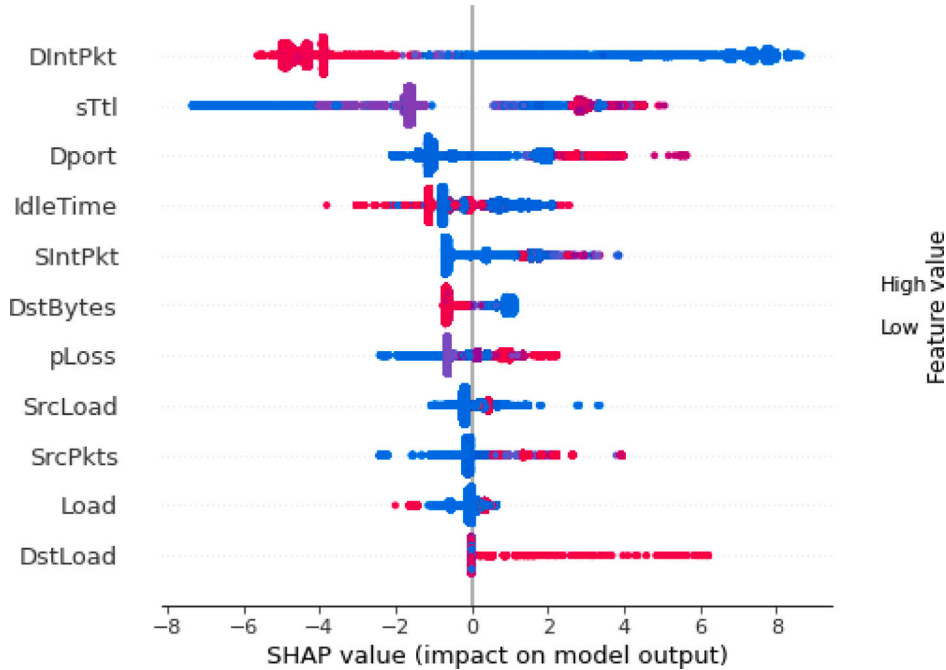


Fig. 7. SHAP values summary plot for the selected features.

5. Model's explainability

Explainability of machine learning models increases trust in the decisions made by these models. In addition, it eliminates “black-box” operation of classifiers, and ensures that the high accuracy achieved by the classifier originates from explainable conditions.

To explain our trained model, in this research we use Shapley additive explanations. SHAP was first introduced in 2017 [29]. Being a model-agnostic explainer gives it the ability to provide general explanations whatever was the type of the classifier used. The method is based on Shapley values first introduced in game theory. Shapley values are based around calculating the impact of each player in a team by calculating the difference between the team's performance with the player, and without the player. This helps in understanding the contribution of the individual player to the team's performance. In explaining our model, SHAP values measure the impact of each feature by measuring the model's performance with and without the feature. This helps in understanding the contribution of the feature to the inference output. The SHAP explainer type used in our experiment was TreeExplainer.

Fig. 7 shows the SHAP values summary plot of the selected 11 features, for the DT classifier. These 11 features are ordered in descending order from the feature with the highest impact on the decision to the lowest.

In **Fig. 7**, the dots shown on the left side of the axis are the feature values that drag the prediction value down, which makes the prediction closer to “benign”, while the values on the right side of the axis pushes the prediction value up which makes the prediction closer to “intrusion”. The dots in red represent a high value of the feature, while the blue dots represent a low value of the feature.

Table 9
Comparison of the performance of the proposed system to previous works.

Literature	Dataset	Features	Classifier	Accuracy	F_1 score	Testing time	Explainable
[23]	WUSTL-IIOT-2018	25	RF	0.9999	–	–	–
[16]	WUSTL-IIOT-2018	19	RF	0.9989	–	–	–
[17]	[17]	–	Extratrees	0.99	0.97	–	–
			RF	0.99	0.97	–	–
			K-NN	0.91	0.89	–	–
[19]	WUSTL-IIOT-2018	7	DT	0.9999	0.9999	–	–
[21]	WUSTL-IIOT-2021	41	MLP	0.9994	0.9994	–	✓
[22]	WUSTL-IIOT-2021	41	FNN-Focal	0.9894	0.6848	–	–
[18]	NSL-KDD and UNSW-NB15	41	DNN	0.989	0.989	–	–
E2I3DS	WUSTL-IIOT-2021	11	DT	0.9997	0.9997	0.1517 μ s	✓

As shown in Fig. 7, DIntPkt features has the highest impact on the prediction. DIntPkt feature represents the destination's inter-packet arrival time in milliseconds. The red dots on the left side indicate that the higher the value of DIntPkt goes, the closer the prediction gets to “benign”, while lower values of this feature pushes the decision closed to “attack”. This is aligned with the fact that most benign data transmission is not done in large bulks sent in a continuous stream of packets. On the other hand, lower times between the packets, indicate a “rapid-fire” mode that is usually used in attacks such as DoS, and port scanning.

The second feature in the figure is sTtl which represents the source-to-destination TTL value. As the figure indicates, low-to-intermediate values could indicate to “benign” traffic, while high values would indicate an “attack”. This is aligned with the fact that most tools used by the attackers create these crafted packets with high TTL value trying to find how deep the attacker can go inside the network. On the other hand, benign traffic would use default TTL values that most applications use. A similar behavior can be seen in PLoss and SrcPkts features. PLoss feature, which indicates the percentage of packets dropped or re-transmitted, shows that high percentage might be an indicator of an attack. This is aligned with the fact that attackers would not be using complete sessions of communications when they are at the reconnaissance stage. This let us the target device assume that the packets were not delivered and re-transmit them, although they have been delivered but the attacker chooses not to respond to them. In a similar manner, SrcPkts, which indicates the number of packets sent from the source to the destination, indicates an attack when its value is high. This is caused by the attacker's attempt to find as many open ports as possible to find an exploitable vulnerability. Another feature connected to this phenomenon is SIntPkt, which indicates the source inter-packet arrival time. This feature indicates an attack with lower values because the more packets the device receives, the inter-packet time would logically decrease.

IdleTime feature indicates an attack when in low value. This is aligned with the fact that attackers conduct consecutive packet transmissions to try to brute-force into a device, put it out of service through DoS, or scan its open ports. Another feature that causes a similar behavior is DstBytes, which indicates the number of bytes sent as a response from the destination to the source. When DstBytes has a low value, this can be indicative of an attack because the target device will re-transmit packets when the attacker does not acknowledge receiving the packets when performing port scanning, or host scanning.

Dport feature, which indicates the destination port number, has a specific range of values that is high to medium that would indicate an attack. The reason behind this mix is that destination ports are usually indicative of the target application layer protocol. Hence, attackers would use specific protocols to conduct their attacks such as Telnet, on port 23, and HTTP, on port 80.

The remaining features, Load, SrcLoad, DstLoad are all relevant to the load measured in bits per second. While high destination load indicates and attack, high source load indicates benign traffic. This is explainable by the fact the most attacks use small payloads and trigger larger responses. This difference makes the general Load feature distributed between high values indicating an attack, and other high values indicating benign traffic.

6. Discussion

The proposed system was extensively tested at multiple development levels to ensure high accuracy, efficient performance, and generalization beyond the training dataset. The proposed system delivered very high accuracy of 0.9997 in its worst case, with a minimal FP rate of 0.0001, and FN rate of 0.0004. These results, along with the results of the cross-validation, indicate that the proposed system has achieved it's first design goal; high accuracy.

Efficiency was improved by the use of DT classifier that required 0.1517 μ s per instance during the testing experiments. This testing time is significantly lower than the time needed by RF classifier that delivered comparable accuracy. In addition, the significant reduction in the number of features, from 41 to 11, contributes to a significant improvement of efficiency by improving the prediction time, data acquisition time, and the feature extraction time.

For the purposes of ensuring generalization, multiple steps were taken. At the earliest stage, all of the host-relevant features, such as the source and destination addresses, were removed to make sure that the classifier does not rely on these host-specific features in making the prediction. In addition, 10-fold cross-validation was performed which produces minimal standard deviation.

Table 9 shows performance metrics comparison with previous works employing machine-learning in intrusion detection in IIoT.

As we examined [16,23], and [19] further, we found out that the dataset used, WUSTL-IIOT-2018, was suffering from strong imbalance where the “benign” class was about 93.93% of the whole dataset. This imbalance was not addressed in all of these papers. Such imbalance would cause the classifier to misbehave outside of the training subset. In addition, the older version of the dataset WUSTL-IIOT-2018, covered a smaller number of attack types in comparison to WUSTL-IIOT-2021 used in our research. While [18] also presented promising accuracy and F_1 score, it was not tested on IIoT datasets. Both UNSW-NB15, and NSL-KDD datasets are general intrusion detection dataset that are not representative of the traffic or attacks performed on IIoT devices, in addition to being outdated.

Prediction time, i.e. the time required to process the input data and produce a classification decision, was measured within the implementation environment. The specifications of the implementation environment were mentioned in Section 4.1. Whether these results are representative of the actual implementation environment or not is dependant on the implementation scenario. If the system is to be deployed at the border of the IIoT network, then the obtained results could be considered representative of the actual processing time required. However, if the deployment was done on the IIoT endpoints, then these measures will not be considered representative of the expected performance because the memory and processing power available in IIoT device are usually much lower than our experimental implementation environment. In general, the prediction time was not published in any of the studies we found, it was not possible to compare proposed system timing metrics with other research. However, in terms of efficiency, the proposed system uses a smaller number of features in comparison to [23], and [16].

The explanation of the classifier’s behavior according to SHAP values showed that it is highly aligned with recognizable attack behavior of malicious actors, and aligned with the current attacking techniques. While explainability was presented in [21] as well, the highest ranking features in SHAP values did not identically match the ones selected in our proposed system. This is due to the fact that in [21], all of the 41 features were used to train and test the neural networks-based system, while our research utilizes RFE-based on feature importance to eliminate the least important features gradually.

7. Conclusions and future directions

In this paper, we presented an efficient explainable machine-learning based intrusion detection system for IIoT, referred to as E2I3DS. The proposed system utilized WUSTL-IIOT-2021 dataset in its most recent version to train a pipeline of machine learning classifiers to choose the best performing one. The number of features was reduced from 41 to 11 using recursive feature elimination based on feature importance. This resulted in increased efficiency while maintaining very high accuracy of 0.9997 for the DT classifier. The proposed classifier underwent 10-fold cross-validation to ensure that it generalizes well beyond its training dataset. These tests delivered minimal standard deviation in accuracy of 0.000072. This established that the classifier does generalize well. In terms of efficiency, the proposed system achieved a prediction time of 0.1517 μ s only, which is a significant improvement in comparison to similar systems.

Future directions of our research can be focused on deployment of the trained model on IIoT devices to measure their performance, further improvement in efficiency, and creating an proper update mechanism for the deployed version of the classifier. Another research direction would be to utilize multi-class classifiers to be able to identify the specific attack type, rather than binary classification.

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.compeleceng.2023.108732>.

Data availability

Data will be made available on request

Appendix. Dataset features

See Table A.10.

Table A.10
WUSTL-IIOT-2021 dataset features [27].

Features	Type	Descriptions
Mean flow (mean)	Float	The average duration of the active flows
Source Port (Sport)	Integer	Source port number
Destination Port (Dport)	Integer	Destination port number
Source Packets (Spkts)	Integer	Source/Destination packet count
Destination Packets (Dpkts)	Integer	Destination/Source packet count
Total Packets (Tpks)	Integer	Total transaction packet count
Source Bytes (Sbytes)	Integer	Source/Destination bytes count
Destination Bytes (Dbytes)	Integer	Destination/Source bytes count
Total Bytes (TBytes)	Integer	Total transaction bytes count
Source Load (Sload)	Float	Source bits per second
Destination Load (Dload)	Float	Destination bits per second
Total Load (Tload)	Float	Total bits per second
Source Rate (Srate)	Float	Source packets per second
Destination Rate (Drate)	Float	Destination packets per second
Total Rate (Trate)	Float	Total packets per second
Source Loss (Sloss)	Float	Source packets retransmitted/dropped
Destination Loss (Dloss)	Float	Destination packets retransmitted/dropped
Total Loss (Tloss)	Float	Total packets retransmitted/dropped
Total Percent Loss (Ploss)	Float	Percent packets retransmitted/dropped
Source Jitter (ScrJitter)	Float	Source jitter in millisecond
Destination Jitter (DrcJitter)	Float	Destination jitter in millisecond
Source Interpacket (SIntPkt)	Float	Source interpacket arrival time in millisecond
Destination Interpacket (DIntPkt)	Float	Destination interpacket arrival time in millisecond
Protocol (Proto)	Char	transaction protocol
Duration(Dur)	Integer	record total duration
TCP RTT (TcpRtt)	Float	TCP connection setup round-trip time, the sum of 'synack' and 'ackdat'.
Idle Time (Idle)	Float	time since the last packet activity. This value is useful in real-time processing, and is the current time - last time.
Sum (sum)	Integer	total accumulated duration of aggregated records
Min (min)	Integer	minimum duration of aggregated records
Max (max)	Integer	maximum duration of aggregated records
Source Diff Serve Byte (sDSb)	Integer	Source different serve byte value
Source TTL (sTtl)	Float	Source → Destination TTL value
Destination TTL (dTtl)	Float	Destination → Source TTL value
Source App Byte (SAppBytes)	Integer	Source → Destination application bytes
Destination App Byte (DAppBytes)	Integer	Destination → Source application bytes
Total App Byte (TotAppByte)	Integer	total application bytes
SYN_Ack (SynAck)	Float	TCP connection setup time, the time between the SYN and the SYN_ACK packets
Run Time (RunTime)	Float	total active flow run time. This value is generated through aggregation, and is the sum of the records duration.
Source TOC (sTos)	Integer	source TOS byte value
Source Jitter (SrcJitAct)	Float	source idle jitter (mSec)
Destination Jitter (DstJitAct)	Float	destination active jitter (mSec)

References

- [1] Sari A, Lekidis A, Butun I. Industrial networks and IIoT: Now and future trends. In: *Industrial IoT*. Springer; 2020, p. 3–55.
- [2] Global industrial Internet of Things market size 2017–2025 | statista. 2021, <https://www.statista.com/statistics/611004/global-industrial-internet-of-things-market-size>. [Accessed 28 December 2021].
- [3] Ahmed M, Jaidka S, Sarkar NI. Security in decentralised computing, IoT and industrial IoT. In: *Industrial IoT*. Springer; 2020, p. 191–211.
- [4] Mishra P, Varadharajan V, Tupakula U, Pilli ES. A detailed investigation and analysis of using machine learning techniques for intrusion detection. *IEEE Commun Surv Tutor* 2018;21(1):686–728.
- [5] Alani MM. Big data in cybersecurity: A survey of applications and future trends. *J Reliab Intell Environ* 2021;7(2):85–114.
- [6] Alani MM, Miri A. Towards an explainable universal feature set for IoT intrusion detection. *Sensors* 2022;22(15):5690.
- [7] Nimbalkar P, Kshirsagar D. Feature selection for intrusion detection system in Internet-of-Things (IoT). *ICT Express* 2021;7(2):177–81.
- [8] Sarhan M, Layeghy S, Portmann M. Towards a standard feature set for network intrusion detection system datasets. *Mob Netw Appl* 2022;1–14.
- [9] Kim B-K, Kang Y. Abnormal traffic detection mechanism for protecting IIoT environments. In: *2018 international conference on information and communication technology convergence*. IEEE; 2018, p. 943–5.
- [10] Wallis K, Hüffmeyer M, Koca AS, Reich C. Access rules enhanced by dynamic IIoT context. In: *IoTBDs*. 2018, p. 204–11.
- [11] Zolanvari M, Teixeira MA, Jain R. Effect of imbalanced datasets on security of industrial IoT using machine learning. In: *2018 IEEE international conference on intelligence and security informatics*. IEEE; 2018, p. 112–7.
- [12] Yao H, Gao P, Zhang P, Wang J, Jiang C, Lu L. Hybrid intrusion detection system for edge-based IIoT relying on machine-learning-aided detection. *IEEE Netw* 2019;33(5):75–81.
- [13] Abdel-Basset M, Chang V, Hawash H, Chakraborty RK, Ryan M. Deep-IFS: Intrusion detection approach for IIoT traffic in fog environment. *IEEE Trans Ind Inf* 2020.

- [14] Hassan MM, Huda S, Sharmeen S, Abawajy J, Fortino G. An adaptive trust boundary protection for IIoT networks using deep-learning feature-extraction-based semisupervised model. *IEEE Trans Ind Inf* 2020;17(4):2860–70.
- [15] Maharani MP, Daely PT, Lee JM, Kim D-S. Attack detection in fog layer for IIoT based on machine learning approach. In: 2020 international conference on information and communication technology convergence. IEEE; 2020, p. 1880–2.
- [16] Teixeira MA, Zolanvari M, Khan KM, Jain R, Meskin N. Flow-based intrusion detection algorithm for supervisory control and data acquisition systems: A real-time approach. *IET Cyber-Phys Syst Theory Appl* 2021.
- [17] Song Y, Luo W, Li J, Xu P, Wei J. SDN-based industrial internet security gateway. In: 2021 international conference on security, pattern analysis, and cybernetics. IEEE; 2021, p. 238–43.
- [18] Awotunde JB, Chakraborty C, Adeniyi AE. Intrusion detection in industrial Internet of Things network-based on deep learning model with rule-based feature selection. *Wirel Commun Mob Comput* 2021;2021:1–17.
- [19] Vulfin AM, Vasilyev VI, Kuharev SN, Homutov EV, Kirillova AD. Algorithms for detecting network attacks in an enterprise industrial network based on data mining algorithms. *J Phys Conf Ser* 2021;2001:012004.
- [20] Kasongo SM. An advanced intrusion detection system for IIoT based on GA and tree based algorithms. *IEEE Access* 2021;9:113199–212.
- [21] Alani MM, Damiani E, Ghosh U. DeepIIoT: An explainable deep learning based intrusion detection system for industrial IOT. In: 2022 IEEE 42nd international conference on distributed computing systems workshops. IEEE; 2022, p. 169–74.
- [22] Dina AS, Siddique AB, Manivannan D. A deep learning approach for intrusion detection in Internet of Things using focal loss function. *Internet of Things* 2023;100699.
- [23] Zolanvari M, Teixeira MA, Gupta L, Khan KM, Jain R. Machine learning-based network vulnerability analysis of industrial Internet of Things. *IEEE Internet Things J* 2019;6(4):6822–34.
- [24] Da Costa KAP, Papa JP, Lisboa CO, Munoz R, de Albuquerque VHC. Internet of Things: A survey on machine learning-based intrusion detection approaches. *Comput Netw* 2019;151:147–57.
- [25] Saranya T, Sridevi S, Deisy C, Chung TD, Khan MKAA. Performance analysis of machine learning algorithms in intrusion detection system: A review. *Procedia Comput Sci* 2020;171:1251–60.
- [26] Géron A. Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems. O'Reilly Media; 2019.
- [27] Zolanvari M. WUSTL-IIOT-2021. 2021, <http://dx.doi.org/10.21227/yftq-n229>.
- [28] Zolanvari M, Teixeira MA, Gupta L, Khan KM, Jain R. Machine learning-based network vulnerability analysis of industrial Internet of Things. *IEEE Internet Things J* 2019;6(4):6822–34.
- [29] Lundberg SM, Lee S-I. A unified approach to interpreting model predictions. *Adv Neural Inf Process Syst* 2017;30.

Mohammed M. Alani is a research fellow at the Cybersecurity Research Lab in Toronto Metropolitan University, Toronto, Canada. He holds a Ph.D. in Computer Engineering. He has authored multiple books and papers published in international venues. He is a Distinguished ACM speaker, and a senior member of the ACM and IEEE. He also serves in the editorial boards of well-known journals.